МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий (ИИТ)
Кафедра цифровой трансформации (ЦТ)

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ

по дисциплине «Разработка баз данных»

Практическая работа № 6

Студент группы *ИВБО-20-24 Лияскин.В.А*

(подпись)

Ассистент *Шерри.Г.Д*

(подпись)

Отчет представлен «23» октября 2026 г.

Москва 2026 г.

ЗАДАНИЕ №1. Триггеры

Триггер 1: Сложная валидация (BEFORE INSERT на sale_item)

Код тригерной функции и тригера

```
-- Функция валидации позиции продажи
CREATE OR REPLACE FUNCTION validate_sale_item_insert()
RETURNS TRIGGER AS $$
DECLARE
    v_stock_qty INT;
    v_product_price DECIMAL(10, 2);
    v_store_id INT;
BEGIN
    -- Получаем ID магазина через сотрудника (в sale нет store_id)
    SELECT e.store_id INTO v_store_id
    FROM employee e
    JOIN sale s ON s.employee_id = e.id
    WHERE s.id = NEW.sale_id;

    -- Получаем остаток на складе конкретного магазина и цену товара
    -- FOR UPDATE блокирует строку от параллельных изменений
    SELECT st.quantity, p.retail_price
    INTO v_stock_qty, v_product_price
    FROM stock st
    JOIN product p ON st.product_id = p.id
    WHERE st.product_id = NEW.product_id
      AND st.store_id = v_store_id
    FOR UPDATE;

    -- Если товара нет на складе этого магазина
    IF NOT FOUND THEN
        RAISE EXCEPTION 'Товар ID=% отсутствует на складе магазина ID=%',
            NEW.product_id, v_store_id;
    END IF;

    -- Проверка достаточности остатка
    IF v_stock_qty < NEW.quantity THEN
        RAISE EXCEPTION 'Недостаточно товара на складе. Доступно: %, запрошено: %',
            v_stock_qty, NEW.quantity;
    END IF;

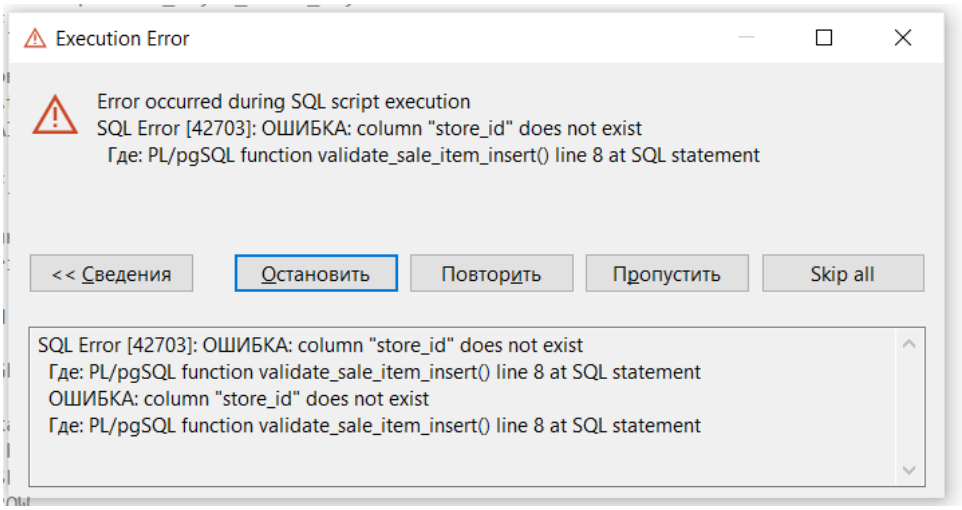

    -- Принудительно устанавливаем актуальную розничную цену
    NEW.price := v_product_price;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- Привязка триггера
CREATE OR REPLACE TRIGGER trg_before_sale_item_insert
BEFORE INSERT ON sale_item
FOR EACH ROW
EXECUTE FUNCTION validate_sale_item_insert();
```

Тест 1: Попытка продать больше, чем есть на складе

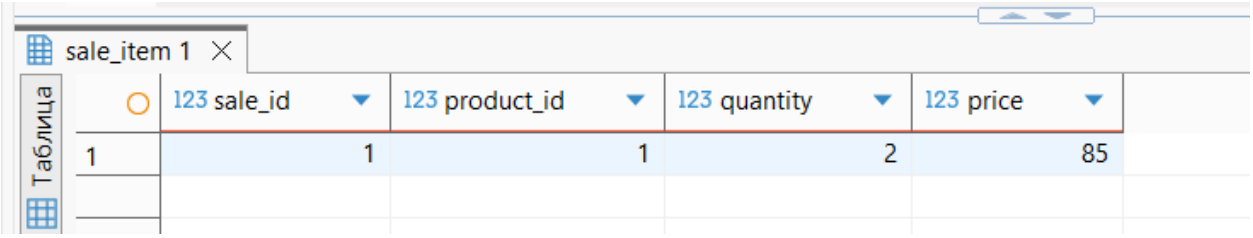
```
INSERT INTO sale_item(sale_id, product_id, quantity, price)
VALUES (13, 1, 999, 85.00);
```



Тест 2: Успешный тест

```
-- Тест 2: успешная продажа
INSERT INTO sale_item(sale_id, product_id, quantity, price)
VALUES (12, 1, 5, 85.00);
```

```
-- Проверяем, что запись появилась
SELECT * FROM sale_item
WHERE sale_id = 1 AND product_id = 1;
```



Проверка остатка прошла успешно - товара на складе хватило

Триггер сам подставил цену - в поле price стоит 85

Запись успешно вставлена в sale_item

Триггер 2: Поддержание согласованности

```
    -- 2. Пересчитываем итоговую сумму продажи
    SELECT COALESCE(SUM(quantity * price), 0)
    INTO sale_total
    FROM sale_item
    WHERE sale_id = target_sale_id;

    -- 3. Обновляем total_amount в шапке sale
    UPDATE sale
    SET total_amount = sale_total
    WHERE id = target_sale_id;

    -- Возвращаемое значение для AFTER-триггера игнорируется,
    -- но синтаксически обязательно
    IF TG_OP = 'DELETE' THEN
        RETURN OLD;
    ELSE
        RETURN NEW;
    END IF;
END;
$$ LANGUAGE plpgsql;

-- Привязка триггера
CREATE OR REPLACE TRIGGER trg_after_sale_item_change
AFTER INSERT OR UPDATE OR DELETE ON sale_item
FOR EACH ROW
EXECUTE FUNCTION update_aggregates_after_sale_change();
```

```
-- Функция поддержания согласованности после изменения позиций продажи
CREATE OR REPLACE FUNCTION update_aggregates_after_sale_change()
RETURNS TRIGGER AS $$
DECLARE
    sale_total DECIMAL(12, 2);
    target_sale_id INT;
    v_store_id INT;
BEGIN
    -- Определяем целевую продажу и магазин
    IF TG_OP = 'DELETE' THEN
        target_sale_id := OLD.sale_id;
    ELSE
        target_sale_id := NEW.sale_id;
    END IF;

    SELECT store_id INTO v_store_id
    FROM sale WHERE id = target_sale_id;

    -- 1. Корректируем склад в зависимости от типа операции
    IF TG_OP = 'DELETE' THEN
        -- Возвращаем товар на склад
        UPDATE stock
        SET quantity = quantity + OLD.quantity
        WHERE product_id = OLD.product_id AND store_id = v_store_id;
    ELSE
        IF TG_OP = 'INSERT' THEN
            -- Списываем товар со склада
            UPDATE stock
            SET quantity = quantity - NEW.quantity
            WHERE product_id = NEW.product_id AND store_id = v_store_id;
        ELSIF TG_OP = 'UPDATE' THEN
            -- Корректируем на разницу
            UPDATE stock
            SET quantity = quantity - (NEW.quantity - OLD.quantity)
            WHERE product_id = NEW.product_id AND store_id = v_store_id;
        END IF;
    END IF;
```

Тест 1: INSERT - списание со склада и пересчёт суммы

```
INSERT INTO sale_item(sale_id, product_id, quantity, price)
VALUES (13, 1, 10, 85.00);
```

Тест 2: DELETE - возврат товара и пересчёт суммы

```sql
DELETE FROM sale_item
WHERE sale_id = 13 AND product_id = 1;
```

Триггер 3: Аудит и логирование

При изменении номера телефона клиента сохранять старое значение в таблицу аудита client_phone_audit.

Код создания таблицы для хранения истории изменений телефонов

```sql
-- Создаём таблицу для хранения истории изменений телефонов
CREATE TABLE IF NOT EXISTS client_phone_audit (
    audit_id INT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    client_id INT NOT NULL,
    old_phone VARCHAR(20) NOT NULL,
    new_phone VARCHAR(20) NOT NULL,
    changed_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    changed_by VARCHAR(50) DEFAULT current_user,
    CONSTRAINT fk_audit_client FOREIGN KEY (client_id)
        REFERENCES client(id) ON DELETE CASCADE
);

COMMENT ON TABLE client_phone_audit IS 'Журнал изменений номеров телефонов клиентов';
```

Код триггерной функции

```sql
-- Функция логирования изменений телефона клиента
CREATE OR REPLACE FUNCTION log_client_phone_change()
RETURNS TRIGGER AS $$
BEGIN
    -- Проверяем, изменился ли именно телефон
    -- IS DISTINCT FROM корректно работает с NULL
    IF NEW.phone IS DISTINCT FROM OLD.phone THEN
        INSERT INTO client_phone_audit (client_id, old_phone, new_phone, changed_at)
        VALUES (
            OLD.id,
            OLD.phone,
            NEW.phone,
            CURRENT_TIMESTAMP
        );
    END IF;

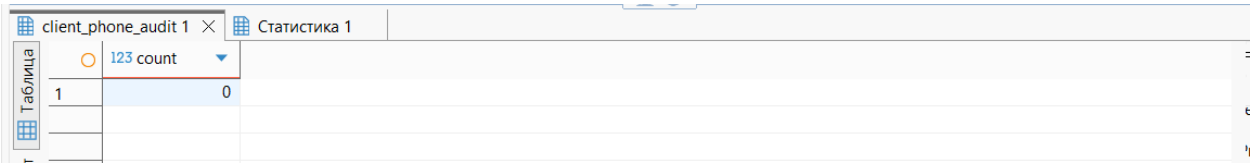
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- Привязка триггера
CREATE OR REPLACE TRIGGER trg_log_client_phone_change
AFTER UPDATE ON client
FOR EACH ROW
EXECUTE FUNCTION log_client_phone_change();
```

Тест 1: Меняем email (тригер не должен сработать)

```sql
-- состояние до в client_phone_audit 0 записей
UPDATE client
SET email = 'ivanov_new@mail.ru'
WHERE id = 1;

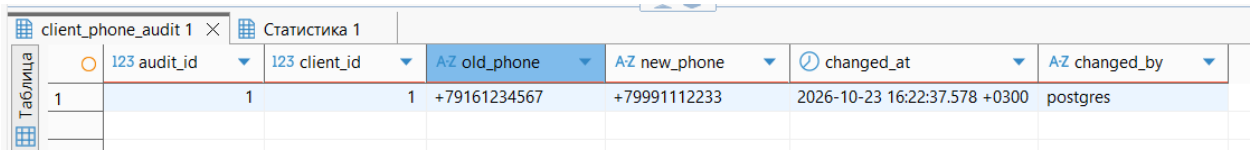
SELECT COUNT(*) FROM client_phone_audit;
-- результат 0,триггер не сработал, потому что телефон не менялся
```



Тест 2: Меняем телефон (триггер должен сработать)

```sql
UPDATE client
SET phone = '+79991112233'
WHERE id = 1;

-- проверяем аудит
SELECT * FROM client_phone_audit ORDER BY audit_id DESC;
```



Результат: 1 запись с old_phone=79161234567, new_phone=79991112233

ЗАДАНИЕ №2. Курсоры

2.1 Явный курсор (DECLARE/OPEN/FETCH/CLOSE)

Задача: Вывести список товаров с их остатками по всем магазинам в формате "Название - Магазин - Количество".

```sql
-- Явный курсор: построчный вывод товаров и их остатков
DO $$
DECLARE
    -- Объявляем курсор, связанный с запросом
    cur_products CURSOR FOR
        SELECT
            p.name AS product_name,
            s.name AS store_name,
            st.quantity
        FROM product p
        JOIN stock st ON p.id = st.product_id
        JOIN store s ON st.store_id = s.id
        ORDER BY p.name, s.name;

    -- Переменная для хранения текущей строки
    row_var RECORD;
    v_count INT := 0;
BEGIN
    -- Открываем курсор (выполняется запрос)
    OPEN cur_products;

    -- Цикл построчного извлечения
    LOOP
        FETCH cur_products INTO row_var;

        -- Если строк больше нет — выходим из цикла
        EXIT WHEN NOT FOUND;

        v_count := v_count + 1;
```

```
        -- Выводим в консоль
        RAISE NOTICE '%. % — %: % шт.',
            v_count,
            row_var.product_name,
            row_var.store_name,
            row_var.quantity;
    END LOOP;

    -- Закрываем курсор (освобождаем ресурсы)
    CLOSE cur_products;

    RAISE NOTICE 'Всего записей обработано: %', v_count;
END;
$$ LANGUAGE plpgsql;
```

DECLARE cur_products CURSOR FOR SELECT... - связываем курсор с запросом

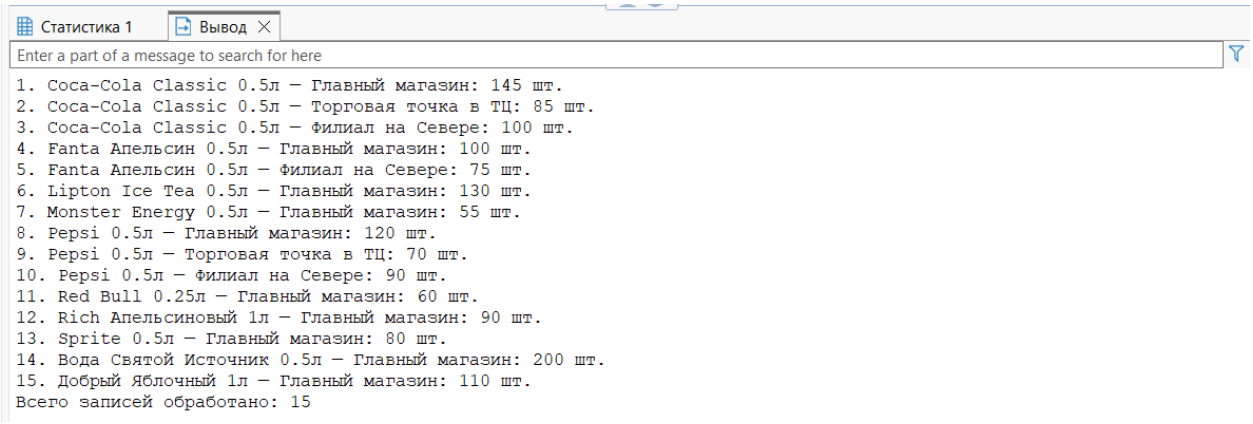
OPEN cur_products - выполняем запрос, курсор встаёт перед первой строкой

FETCH cur_products INTO row_var - получаем следующую строку в переменную

EXIT WHEN NOT FOUND - выход из цикла, когда строки закончились

CLOSE cur_products - освобождаем ресурсы

Вывод консоли



2.2 Неявный курсор (цикл FOR...IN) с возвратом таблицы

Задача: Создать функцию, которая возвращает список клиентов с их картами лояльности и текущим уровнем скидки.

```sql
-- Функция с неявным курсором, возвращающая таблицу
CREATE OR REPLACE FUNCTION get_clients_with_loyalty()
RETURNS TABLE (
    client_id_out INT,
    full_name_out TEXT,
    phone_out VARCHAR(20),
    card_number_out VARCHAR(16),
    loyalty_level_out VARCHAR(30),
    discount_out NUMERIC(5, 2)
)
LANGUAGE plpgsql AS $$
DECLARE
    client_row RECORD;
BEGIN
    -- Цикл FOR...IN — неявный курсор
    -- PostgreSQL сам делает OPEN, FETCH, CLOSE
    FOR client_row IN
        SELECT
            c.id,
            COALESCE(
                (c.full_name).last_name || ' ' ||
                (c.full_name).first_name ||
                COALESCE(' ' || (c.full_name).middle_name, ''),
                'Анонимный клиент'
            ) AS full_name,
            c.phone,
            lc.number AS card_number,
            ll.name AS level_name,
            ll.discount
        FROM client c
        LEFT JOIN loyalty_card lc ON c.id = lc.client_id
        LEFT JOIN loyalty_level ll ON lc.level_id = ll.id
        ORDER BY c.id
    LOOP

        -- Заполняем выходные столбцы
        client_id_out := client_row.id;
        full_name_out := client_row.full_name;
        phone_out := client_row.phone;
        card_number_out := client_row.card_number;
        loyalty_level_out := client_row.level_name;
        discount_out := client_row.discount;

        -- Добавляем строку в результат функции
        RETURN NEXT;
    END LOOP;

    RETURN;
END;
$$;

-- Вызов функции
SELECT * FROM get_clients_with_loyalty();
```

RETURNS TABLE (...) - функция возвращает набор строк с указанной структурой

FOR client_row IN SELECT... - неявный курсор. PostgreSQL сам:

Объявляет курсор

Открывает его

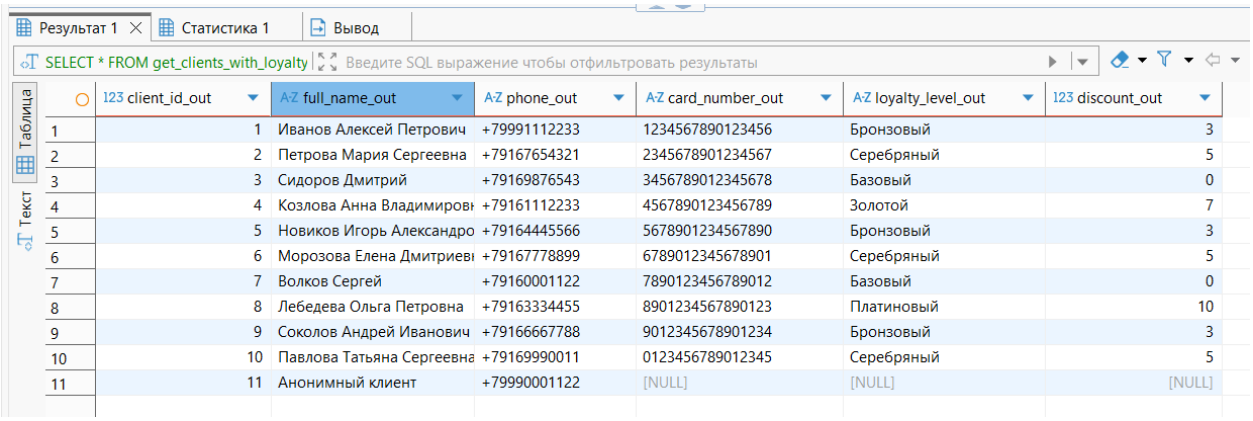
Извлекает строки по одной в client_row

Закрывает после завершения цикла

RETURN NEXT - добавляет текущую строку в результирующий набор

RETURN в конце - завершает функцию

Вывод



Результат 1 | Статистика 1 | Вывод

SELECT * FROM get_clients_with_loyalty — Введите SQL выражение чтобы отфильтровать результаты

Таблица | Текст

	client_id_out	full_name_out	phone_out	card_number_out	loyalty_level_out	discount_out
1	1	Иванов Алексей Петрович	+79991112233	1234567890123456	Бронзовый	3
2	2	Петрова Мария Сергеевна	+79167654321	2345678901234567	Серебряный	5
3	3	Сидоров Дмитрий	+79169876543	3456789012345678	Базовый	0
4	4	Козлова Анна Владимирови	+79161112233	4567890123456789	Золотой	7
5	5	Новиков Игорь Александро	+79164445566	5678901234567890	Бронзовый	3
6	6	Морозова Елена Дмитриеви	+79167778899	6789012345678901	Серебряный	5
7	7	Волков Сергей	+79160001122	7890123456789012	Базовый	0
8	8	Лебедева Ольга Петровна	+79163334455	8901234567890123	Платиновый	10
9	9	Соколов Андрей Иванович	+79166667788	9012345678901234	Бронзовый	3
10	10	Павлова Татьяна Сергеевна	+79169990011	0123456789012345	Серебряный	5
11	11	Анонимный клиент	+79990001122	[NULL]	[NULL]	[NULL]